

M1 architecture - current snapshot

5 October 2026 | Specified design | Implementation validation pending

The system

- **Main flow:** intake -> intent capsule -> intent Gate -> requirement capsules -> requirement Gate -> planner -> validated DAG -> capsule/Gate binding and freeze -> local execution -> delivery.
- SwarmFlow provides the fixed required outer flow. Planned task nodes also become fixed after validation, binding and freeze.
- The planner creates nodes, dependencies and typed data bindings from accepted requirements and an admitted library snapshot. It is an orchestration service, not a capsule.
- Task data enters the DAG through declared inputs. Dispatch executes ready nodes using their pinned local capsules. Every capsule call is followed by its bound Gate capsule before dependent work can consume the result.
- Output/capture, Verification and release are durable authority. A failed Gate or failed commit stops following work capsules; recovery reuses committed evidence.
- One Dockerized modular monolith contains the control plane and restricted local execution processes. Model work uses the protected Codex adapter.
- The research chain is a capability example, not the whole system. The previous twelve-capability inventory and 23 research bindings are baseline material; intent identities and revised planner entry contracts still need reconciliation.
- One shared verifier capability serves the Gate call sites through pinned criteria profiles. Every Gate-role capsule has zero RSI-mutable components.
- Delivery is ordinary result processing/publication and returns authorized data to the user view. It is not a capsule.
- Offline RSI cannot change live runs or activate candidates. SkillFuzz interaction analysis remains deferred.

Component roles

- **Capability capsule (CC):** admitted, versioned capability with a Declaration, typed ports, dependencies, effects and checks. It is stored for planning and application and is not tied to one task. See Declaration.
- **Workflow node:** the hot-path use of a CC for one specific task, bound into the workflow with actual input references, limits and its Gate. Multiple nodes can use one capability. See run plan.
- **Ordinary module:** control or deterministic helper without independent capsule admission. Examples: supervisor, extraction, resource freezing, numeric arithmetic, plan validator and publisher. See pipeline.

Workflow order

- SwarmFlow runs the fixed outer workflow. Its required preparation, planning, dispatch and delivery positions remain present; planned nodes become fixed at freeze.
- Intake preserves the request and permitted resource snapshots.
- One intent compilation capsule derive intent. Each runs through its associated intent Gate; accepted intent is required before requirements begin.

Read in Obsidian: [presentation/showcase/README.md](#)

Existing research capability baseline

Twelve existing research, verifier and operator identities. Intent capabilities and revised planner entry are not yet reconciled; this is not the complete corrected inventory.

Capability	Responsibility and execution	Dependencies / behavior owner
research.compile_brief	Skill; one semantic pass turns the source request into a research contract	Requirement
research.search_ideas	Tool; bounded model work and retrieval form evidence-backed ideas	Both local and scholarly operators per query; Search
research.select_opportunity	Tool; consolidation and scoring, deterministic filtering/ranking, one selected card	Frozen dependency registry; Screening
research.form_hypothesis	Tool; bounded model work freezes hypotheses, predicates and protocol	CodeSearch and resource freezing; Hypothesis
research.build_poc	Tool; bounded generation creates experiment files and harness	CodeSearch; POC
research.run_benchmark	Tool; no model; coordinates trusted baseline/treatment execution	Measurement adapters and confined worker; Benchmark
research.evaluate_results	Bounded plausibility turn plus deterministic numeric classification	Read-only measurement evidence; Evaluation
research.write_report	Skill; one synthesis turn, then ordinary rendering/publication	No admitted nested dependency; Delivery
research.verifier	Shared skill; stage-selected semantic assessment, not numeric scientific classification	Verifier, assessment
op.local_search	Tool; query and local documents produce ranked search hits	Local search
op.scholarly_search	Tool; query produces bounded scholarly search hits	Scholarly search
op.codesearch	Tool; query and repository snapshot produce code hits	CodeSearch, code hits

Build and improve

- Build storage, runner, admission, broker and minimal fixtures first; then intent capsules and Gates, requirements and Gates, planner/validator/binder, then durable node continuation. Reconcile the old build order before coding. Add retrieval/Screening, Hypothesis/POC, scientific execution/evaluation/report, workstation/export, RSI and isolated experiments in dependency order. Build order owns prerequisites.
- Keep algorithm choices inside modules. Preserve output semantics, frozen policies, permission boundaries and evidence ordering when optimizing.
- Measure model calls separately from deterministic helper work. Use pinned fixtures and compare correctness before latency/cost. Capability guide owns proposed optimization experiments; benchmark materials records candidate material and restrictions.
- SkillFuzz capsule-set interaction analysis is deferred. Current contract checks establish interface compatibility, not semantic safety of every combination. See library.

Runtime, recovery and improvement

5 October 2026 | Specified design | Implementation validation pending

Production authority

- Accepted intent and requirements precede planning. The planner proposes the DAG; the supervisor validates it, binds work/Gate capsules, freezes the library/configuration/policy closure and owns dispatch. The runner enforces a Binding, validates ports, reserves call budgets and captures execution. Capsules cannot choose their successor.
- The model bridge wraps Codex behind a provider interface. Production uses the static configured route; experimental routing selects an allowed endpoint inside a call, not a capability or workflow successor.
- Work output and capture commit first. Gate host evaluates the evidence under a pinned profile and persists Verification through the trusted writer. The supervisor commits release before dispatching the next node.
- Advancement requires committed PASS or PASS_WITH_KNOWN_LIMITATIONS with routing_action: ADVANCE, plus the release record. A passing calculation whose durable save fails cannot release work. See lifecycle.
- Scientific PASS, FAIL, INCONCLUSIVE and CONDITIONALLY_ACCEPTABLE describe research results, not infrastructure permission to advance. A valid negative result can reach a report. See evaluation.

Failure and recovery

- Stable duplicate request IDs reuse committed results or report in-progress state; changed bytes conflict. Model/effectful calls do not silently repeat after uncertain outcomes.
- A failed Gate stops subsequent capsule dispatch for the run, including other ready branches. Existing committed outputs are retained; in-flight work follows cancellation/capture rules. Timeout, cancellation, invalid inputs, denied effects and persistence failures also halt and retain safe evidence. Attribution uses the reason registry, not a different error vocabulary per capsule.
- Recovery inspects committed state: reuse work evidence, reevaluate a missing Gate, reuse a committed decision, or dispatch from an existing release. Explicit environment/partial-effect remediation may create an attempt under unchanged pins. Changed inputs, implementation, policy or configuration require a new run. Lifecycle and storage own exact rules.
- Screening with no eligible card records NO_ELIGIBLE_OPPORTUNITY and halts before Hypothesis for human review; it does not create a successful empty card. See Screening.

Admission and offline RSI

- Admission always checks declaration integrity, hashes, closure and permissions before the selected provider. Tested admission records checks actually executed. Puppet admission allowlists exact developer-selected hashes with exempt assurance; it cannot invent certification or runtime Gate passes.
- Library keeps version, assurance and active alias separate. A candidate can be admitted but inactive; manual activation changes future snapshots, not a frozen run.
- Offline RSI uses the private fixture oracle, bounded session/lifetime budgets and lineage evidence. Permitted changes stay inside the target's frozen contract. It cannot modify schemas, referee, acceptance policy, permissions, live DAG or active aliases.

Read in Obsidian: [presentation/showcase/runtime-and-improvement.md](#)

Data, credentials and access

5 October 2026 | Specified design | Implementation validation pending

Input and output

- Intake validates locally provisioned text, Markdown, PDF, repository and dataset resources, then creates immutable snapshots. Extraction creates source text with source references and offset basis. Intent capsules then derive accepted intent through their Gates; gated requirements supply the planner and DAG input contract. The exact revised ports are tracked in control flow. See intake, source text, resources and workstation.
- Literature enters through bounded scholarly search; local/code search uses declared resources. A model receives authorized read-only evidence projections, not arbitrary store access. See local search, scholarly search and CodeSearch.
- Scientific execution freezes protocol, datasets, baseline/treatment, methods, environment and predicates before execution. Trusted adapters own measurements; generated code supplies experiment behavior rather than self-awarded scores. See measurement protocol.
- Delivery is ordinary code, not a capsule: it processes/formats accepted terminal results and returns published artifacts through authorized retrieval/user views. The report-writing CC is separate from this delivery boundary.
- Publisher input is the report **and** POC, benchmark, trusted StageContext (the frozen stage/run evidence context), destination and request identity. It requires the committed report Verification/release, validates the complete manifest and publishes files atomically. See delivery.
- The external benchmark runner uses authenticated local HTTP invocation/status/abort and sealed manifest/export retrieval. It does not use a runtime file-upload or clone endpoint. The API is published on 127.0.0.1:8787, requires a local bearer token except for liveness, and accepts one active research run. Its final external schema connects through a bounded export adapter; harness internals are separate. See benchmark export.

Evidence

- Immutable artifacts and append-only control records share run/step/attempt/request and policy/configuration references. The trusted writer owns public records; the publisher owns its destination; the private oracle owns hidden fixture evidence. See records and storage.
- Freeze prevents later stages from changing data, scoring policies or dependencies. Content hashes identify exact bytes; aliases are resolved before freeze.
- Required raw capture and durable records remain enabled in every diagram variant. Telemetry/UI projections are derived; hiding observability edges does not disable required capture or change authority. See telemetry and information flow.

Permission boundaries

- Dedicated persistent Codex credential storage has a separate login and one serialized owner. Do not copy a personal desktop auth file into disposable workers. The auth provider is replaceable behind the bridge. Exact refresh/account behavior still needs real tests. See model authentication.
- Generated experiment code runs in a restricted child with declared input/output paths. It receives no model credentials, public store, hidden fixtures, Docker socket or unrestricted network. Containerization alone is not the isolation claim. See process boundary.
- Startup pre-checks fail closed if the selected confinement mechanism is unavailable. Linux container mechanisms and macOS host operation require validation; an unsupported profile cannot count as platform acceptance. See deployment and environment.
- Proposed platform targets are Linux Docker Engine and macOS Docker Desktop running the same pinned Linux/Python 3.12 image. The repository allows Python $\geq 3.11, < 3.14$; that range is not a validated image matrix. Windows native is outside M1 targets. See environment.
- Configuration layers are packaged defaults, user settings, then project settings. Project values override user values within protected policy limits. Changes affect a new run; resume retains its frozen snapshot. Service-affecting settings require an explicit restart with no active run. Dependencies come from a pinned offline wheelhouse. See environment.

Read in Obsidian: [presentation/showcase/data-and-permissions.md](#)

Contracts, planner and coding checks

5 October 2026 | Specified design | Implementation validation pending

Exact interfaces

- Producer and consumer use the exact pinned port type/version. An explicit pinned adapter is required for a mismatch; the binder cannot silently coerce data. Run plan and Declaration own this rule.
- Released versions are immutable. Draft changes update the owner and all affected consumers together; release changes require a new version and migration decision. Controlled ext data does not allow unknown core fields. Policies owns release control.
- Research ports and the 23-binding research baseline are retained baseline contracts. Current M1 control flow requires intent capsule/Gate ports, accepted requirement entry and planner/binder callers to be reconciled before coding handoff.
- Artifact references identify stored immutable evidence, not caller-selected filesystem paths. Validation includes existence, schema version, hash and authorization. Storage owns resolution.

Planning before DAG execution

- The planner is an orchestration service in the corrected main flow, using the configured Codex bridge. Accepted requirements and an admitted library snapshot are its inputs; a candidate DAG is its output. Control flow supersedes the former experimental-only entry.
- The deterministic validator checks exact schemas, closure, cycles, reachability, bounds, permissions, required Gates and objective coverage. Only a committed valid proposal can freeze and dispatch. The corrected main flow plans after the requirement Gate; the former fixed-template bootstrap must be revised.
- Invalid proposals preserve structured findings and cannot dispatch. No live replanning or unrestricted delegation is introduced.
- **Deferred:** SkillFuzz analysis of how CC sets interact. Planned structural/failure fixtures remain; probabilistic interaction screening is not a current admission, planning or handoff prerequisite. No interaction campaign is claimed to have run.

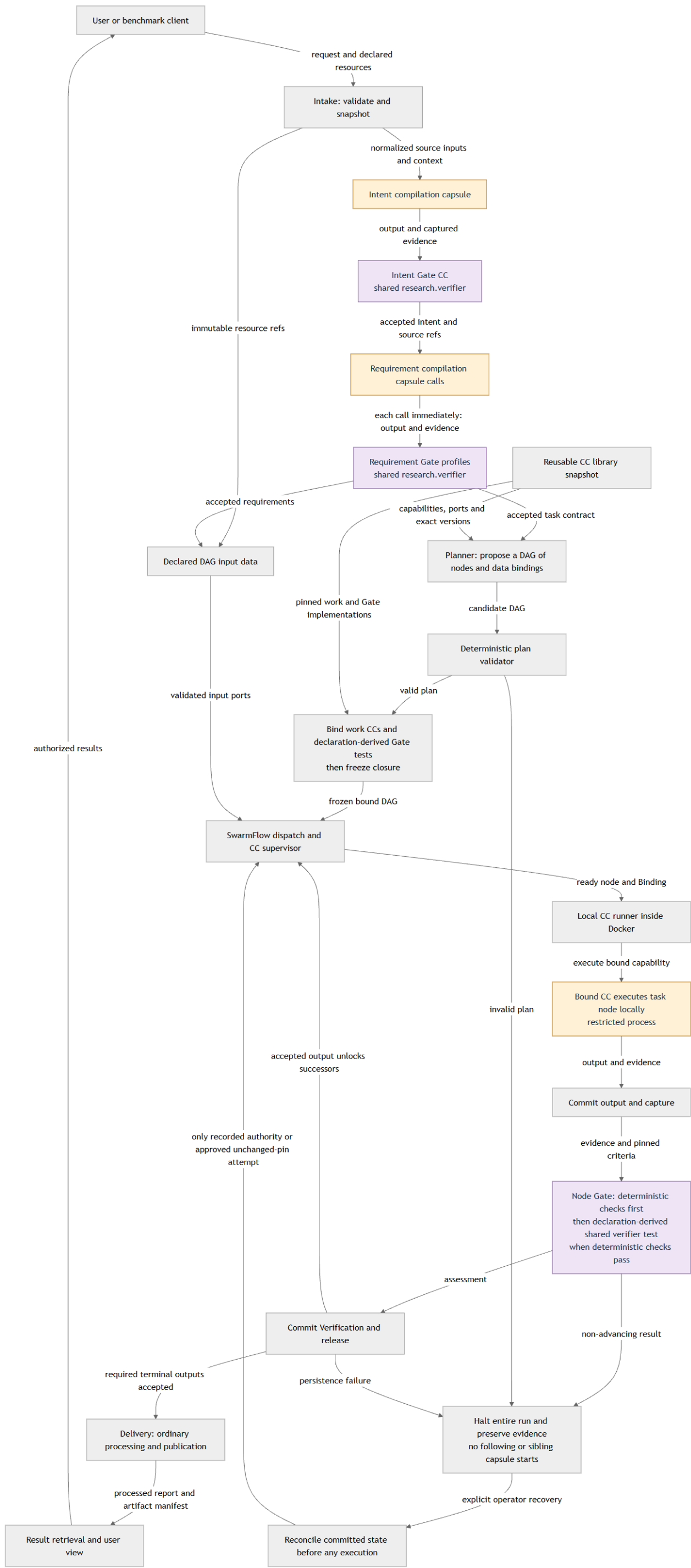
Evidence and development

- **Specified design:** a contract and expected behavior are written at an owning page. Draft does not mean implemented or approved.
- **Documentation checked:** a named command/review checked a particular source revision. Check freshness before reusing the result.
- **Implementation validation pending:** runtime behavior, real platform/security enforcement, authentication, crash durability and measurement accuracy need executed tests after coding.
- Coder requirements asks seven questions: placement, interfaces, control flow, persistence, security, environment and verification.
- Handoff, module map and build order link exact owners and prerequisites. A coder can invoke each boundary independently with fake adjacent services before joining the full workflow.
- Architecture defines contracts and expected observations. The downstream team creates Spec Kit, code, tests and actual results. No measured optimization or complete-system safety claim is supplied by this package.

Read in Obsidian: [presentation/showcase/validation-and-development.md](#)

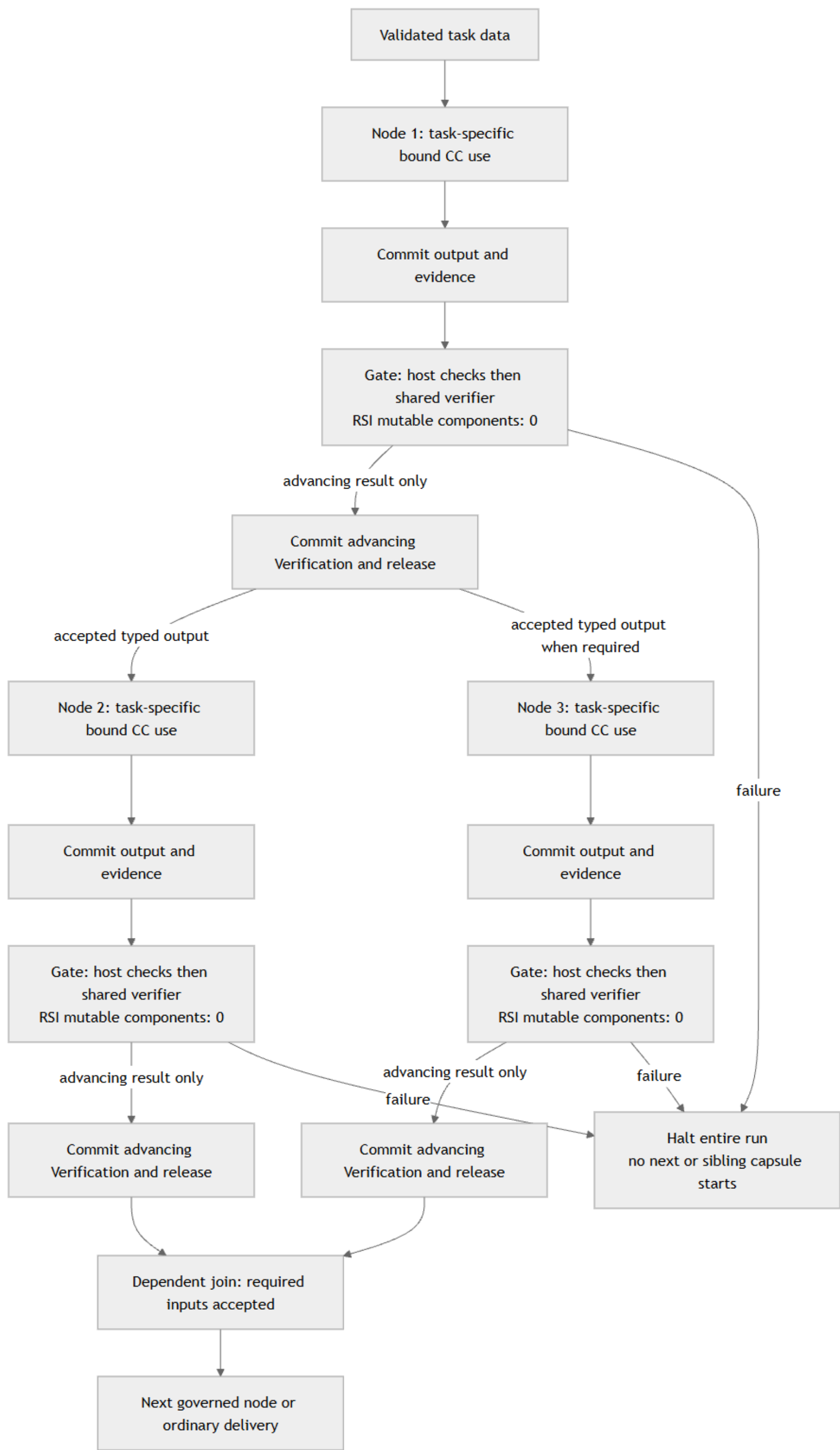
Whole M1 flow: intake to gated DAG execution

Intent and requirements are accepted before planning. Local nodes execute their bound CC then their Gate.



Bound DAG nodes: data and acceptance order

Dependencies unlock only after committed advancing Verification and release. Branches do not promise parallel execution.



Halt, durable advancement and recovery

Per-node ordering remains: output/capture, Gate, Verification, release. Revised frontend contracts remain pending.

